PRACTICAL WORK 4
AI-Based Object Detection and ROS2 Hardware Control

Pretrained YOLO11n | Sports Ball Class Filtering | Hybrid YOLO–HSV | ROS2 Topic Remapping

Participant	Class / Group	Date
_____	_____	_____

1. Aim and Learning Outcomes

Aim: Develop a pretrained YOLO-based object detection system, progressively integrate it into existing ROS2 nodes, and control ESP32 LEDs based on the position of a green sports ball.

LO1. Run a pretrained YOLO model for real-time multi-class object detection.

LO2. Filter detections by COCO class ID 32 (sports ball).

LO3. Apply HSV colour verification to YOLO sports-ball bounding boxes.

LO4. Publish YOLO green-ball coordinates using ROS2 topics and reuse an existing decision node through topic remapping.

LO5. Integrate the AI-based vision pipeline with existing ESP32 control and a multi-node launch file.

2. Prerequisites and System Specification

Item	Configuration
Ubuntu / ROS2	Ubuntu 24.04 / ROS2 Jazzy
Workspace / package	~/mindspark_ws / mindspark_robot
Virtual environment	~/mindspark_ws/yolo_venv (created in Section A)
Model	yolo11n.pt, COCO classes
Camera	USB webcam, 320 × 240; /dev/video0 (device index 0)
ROS Domain	ROS_DOMAIN_ID=30
ESP32 serial	/dev/ttyUSB0; 115200 baud
Outputs	GPIO25 red=LEFT; GPIO26 yellow=CENTER; GPIO27 green=RIGHT
Existing nodes	camera_node, vision_decision_node, esp32_control_node
New PW4 nodes	yolo_detection_node, yolo_sports_ball_node, yolo_green_ball_node

Safety and preparation: Stop PW3 launch processes before opening the webcam. Close Arduino Serial Monitor before using the ESP32 through ROS2. Connect an ESP32-compatible LED module and common ground; check voltage and active-high characteristics.

3. Overall Workflow

Phase 1: standalone general YOLO node → Phase 2: standalone sports-ball-only node → Phase 3: camera ROS2 topic + YOLO–HSV green-ball node → Phase 4: decision-node topic remapping → Phase 5: ESP32 hardware control → Phase 6: single ROS2 launch command.

Core ROS2 data flow: camera_node → /camera/image_raw → yolo_green_ball_node → /yolo/green_ball → vision_decision_node (remapped) → /vision/decision → esp32_control_node → ESP32 LEDs.

A. Prepare the YOLO Python Environment

A1. Verify ROS2 and Python. Open a new terminal:

```
source /opt/ros/jazzy/setup.bash
echo $ROS_DISTRO
python3 --version
```

A2. Create and activate a dedicated Python virtual environment; keep ROS2 system packages accessible:

```
sudo apt install python3-venv python3-pip
cd ~/mindspark_ws
python3 -m venv --system-site-packages yolo_venv
source ~/mindspark_ws/yolo_venv/bin/activate
```

Expected terminal:

```
(yolo_venv) ilmi@ilmi:~/mindspark_ws$
```

A3. Install Ultralytics, keeping the NumPy version constraint for this tested teaching environment:

```
echo 'numpy>=1.26,<2' > ~/mindspark_ws/yolo_constraints.txt
python3 -m pip install -c ~/mindspark_ws/yolo_constraints.txt ultralytics
python3 -m pip install 'setuptools>=30.3.0,<80' cffi
```

A4. Verify dependencies. If pip reports another conflict, record it and resolve it before continuing:

```
python3 -m pip check
python3 -c "import rclpy; from ultralytics import YOLO; print('ROS2 + YOLO: OK')"
python3 -c "import torch; print('CUDA Available:', torch.cuda.is_available())"
```

Observed reference setup: installation succeeded with PyTorch CUDA available. Other computers may use CPU. Do not install YOLO into system Python using sudo pip.

SCREENSHOT EVIDENCE — A: successful environment checks

_____

_____

B. General Object Detection – All COCO Classes

B1. Create file:

```
code ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_detection_node.py
```

B2. Use the complete node below. It directly opens the webcam and does not publish a custom ROS2 topic in Phase 1.

```
#!/usr/bin/env python3
import time
import cv2
import rclpy
from rclpy.node import Node
from ultralytics import YOLO

class YOLODetectionNode(Node):
    def __init__(self):
        super().__init__('yolo_detection_node')
        self.model = YOLO('yolo11n.pt')
        self.cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
        if not self.cap.isOpened():
            raise RuntimeError('Cannot open webcam')
        self.cap.set(cv2.CAP_PROP_FOURCC,
                     cv2.VideoWriter_fourcc(*'MJPG'))
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
```

```
        self.frames = 0
        self.start = time.perf_counter()
        self.fps = 0.0
        self.timer = self.create_timer(0.01, self.detect)
        self.get_logger().info('General YOLO detection started')

    def detect(self):
        ok, frame = self.cap.read()
        if not ok:
            self.get_logger().warning('Camera read failed')
            return
        result = self.model.predict(frame, conf=0.40,
                                    imgsz=640, verbose=False)[0]
        display = result.plot()
        self.frames += 1
        elapsed = time.perf_counter() - self.start
        if elapsed >= 5.0:
            self.fps = self.frames / elapsed
            self.get_logger().info(f'Detection FPS: {self.fps:.2f}')
            self.frames = 0
            self.start = time.perf_counter()
        cv2.putText(display, f'FPS: {self.fps:.1f}', (10, 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)
        cv2.putText(display, f'Objects: {len(result.boxes)}',
                    (10, 45), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 255), 2)
        cv2.imshow('PW4 - YOLO General Object Detection', display)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            self.timer.cancel()
            rclpy.try_shutdown()

    def destroy_node(self):
        self.cap.release()
        cv2.destroyAllWindows()
        super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = YOLODetectionNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

B3. Execute inside the YOLO virtual environment:

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/yolo_venv/bin/activate
export ROS_DOMAIN_ID=30
python3 ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_detection_node.py
```

B4. Verify in a second terminal (no venv required):

```
source /opt/ros/jazzy/setup.bash
export ROS_DOMAIN_ID=30
ros2 node list
ros2 node info /yolo_detection_node
```

Expected / verify: The node appears in the ROS2 graph; bounding boxes, class labels, confidence and FPS appear in the video window.

SCREENSHOT EVIDENCE — B: general YOLO detection and ros2 node list

_____

_____

C. Sports Ball Class Filtering

C1. Create a separate node so the general detection activity remains available:

```
code ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_sports_ball_node.py
```

C2. Full sports-ball-only node. COCO class ID 32 is sports ball. It displays centre (cx, cy).

```
#!/usr/bin/env python3
import time
import cv2
import rclpy
from rclpy.node import Node
from ultralytics import YOLO

class YOLOSportsBallNode(Node):
    def __init__(self):
        super().__init__('yolo_sports_ball_node')
        self.model = YOLO('yolo11n.pt')
        self.cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
        if not self.cap.isOpened():
            raise RuntimeError('Cannot open webcam')
        self.cap.set(cv2.CAP_PROP_FOURCC,
                     cv2.VideoWriter_fourcc(*'MJPG'))
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
        self.cap.set(cv2.CAP_PROP_FPS, 30)
        self.frames = 0
        self.start = time.perf_counter()
        self.fps = 0.0
        self.timer = self.create_timer(0.01, self.detect)

    def detect(self):
        ok, frame = self.cap.read()
        if not ok:
            return
        result = self.model.predict(frame, conf=0.40,
                  imgsz=640, classes=[32], device=0,
                  verbose=False)[0]
        display = result.plot()
        for box in result.boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            cv2.circle(display, (cx, cy), 4, (0, 0, 255), -1)
            cv2.putText(display, f'({cx}, {cy})',
                        (x1, min(y2 + 16, display.shape[0]-5)),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.4,
                        (0, 255, 255), 1)
        self.frames += 1
        elapsed = time.perf_counter() - self.start
        if elapsed >= 5.0:
            self.fps = self.frames / elapsed
            self.get_logger().info(f'Detection FPS: {self.fps:.2f}')
            self.frames = 0
            self.start = time.perf_counter()
        cv2.putText(display, f'FPS: {self.fps:.1f}', (10, 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)
        cv2.putText(display, f'Balls: {len(result.boxes)}',
                    (10, 45), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 255), 2)
        cv2.imshow('PW4 - YOLO Sports Ball Detection', display)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            self.timer.cancel()
            rclpy.try_shutdown()

    def destroy_node(self):
        self.cap.release()
        cv2.destroyAllWindows()
        super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = YOLOSportsBallNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
```

```
        pass
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

C3. Stop Phase 1 before launching Phase 2 (same camera). Run from yolo_venv:

```
python3 ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_sports_ball_node.py
```

C4. Show a person, bottle and two sports balls. Verify that only the 'sports ball' class receives bounding boxes. The pretrained model may miss unusual balls.

SCREENSHOT EVIDENCE — C: sports-ball-only boxes and centre coordinates

_____

_____

D. ROS2 YOLO–HSV Green Ball Detection

D1. Stop the standalone YOLO node. This version subscribes to camera_node images, rejects non-green balls and publishes a text message compatible with the PW3 decision node.

```
code ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_green_ball_node.py
```

D2. Full `yolo_green_ball_node.py` (same tested architecture and interface):

```
#!/usr/bin/env python3
import time
import cv2
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
from ultralytics import YOLO

class YOLOGreenBallNode(Node):
    def __init__(self):
        super().__init__('yolo_green_ball_node')
        self.model = YOLO('yolo11n.pt')
        self.bridge = CvBridge()
        self.frame_width = 320
        self.frame_height = 240
        self.lower_green = np.array([35, 70, 50], dtype=np.uint8)
        self.upper_green = np.array([85, 255, 255], dtype=np.uint8)
        self.green_threshold = 0.30
        self.latest_frame = None
        self.new_frame_available = False
        self.frames = 0
        self.start = time.perf_counter()
        self.fps = 0.0
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10)
        self.publisher = self.create_publisher(
            String, '/yolo/green_ball', 10)
        self.timer = self.create_timer(0.01, self.process_frame)
        self.get_logger().info('YOLO Green Ball Detection Started')

    def image_callback(self, msg):
        try:
            frame = self.bridge.imgmsg_to_cv2(
                msg, desired_encoding='bgr8')
            if frame.shape[:2] != (self.frame_height, self.frame_width):
                frame = cv2.resize(frame, (self.frame_width,
                                           self.frame_height))
            self.latest_frame = frame
            self.new_frame_available = True
        except Exception as error:
```

```
            self.get_logger().error(f'Image conversion error: {error}')

    def green_ratio(self, frame, x1, y1, x2, y2):
        roi = frame[y1:y2, x1:x2]
        if roi.size == 0:
            return 0.0
        hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
        mask = cv2.inRange(hsv, self.lower_green, self.upper_green)
        h, w = mask.shape
        circular = np.zeros((h, w), dtype=np.uint8)
        cv2.circle(circular, (w // 2, h // 2),
                   max(1, min(w, h) // 2), 255, -1)
        green = cv2.countNonZero(cv2.bitwise_and(mask, circular))
        total = cv2.countNonZero(circular)
        return green / total if total else 0.0

    def process_frame(self):
        if self.latest_frame is None or not self.new_frame_available:
            return
        frame = self.latest_frame.copy()
        self.new_frame_available = False
        result = self.model.predict(frame, conf=0.40,
                  imgsz=640, classes=[32], device=0,
                  verbose=False)[0]
        display = frame.copy()
        h, w = frame.shape[:2]
        best = None
        best_conf = -1.0
        count = 0
        for box in result.boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            x1, x2 = max(0, min(x1, w)), max(0, min(x2, w))
            y1, y2 = max(0, min(y1, h)), max(0, min(y2, h))
            if x2 <= x1 or y2 <= y1:
                continue
            confidence = float(box.conf[0].item())
            ratio = self.green_ratio(frame, x1, y1, x2, y2)
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            if ratio >= self.green_threshold:
                count += 1
                cv2.rectangle(display, (x1, y1),
                              (x2, y2), (0, 255, 0), 2)
                cv2.circle(display, (cx, cy), 4, (0, 0, 255), -1)
                cv2.putText(display,
                            f'GREEN BALL {confidence:.2f} G:{ratio:.0%}',
                            (x1, max(15, y1-8)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.4,
                            (0, 255, 0), 1)
                cv2.putText(display, f'({cx}, {cy})',
                            (x1, min(y2+15, h-5)),
                            cv2.FONT_HERSHEY_SIMPLEX,
                            0.4, (0, 255, 255), 1)
                if confidence > best_conf:
                    best_conf = confidence
                    best = (cx, cy)
            else:
                cv2.rectangle(display, (x1, y1),
                              (x2, y2), (0, 0, 255), 1)
                cv2.putText(display, f'REJECTED G:{ratio:.0%}',
                            (x1, max(15, y1-8)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.4,
                            (0, 0, 255), 1)
        msg = String()
        msg.data = (f'DETECTED: x={best[0]}, y={best[1]}'
                    if best is not None else 'NOT DETECTED')
        self.publisher.publish(msg)
        self.frames += 1
        elapsed = time.perf_counter() - self.start
        if elapsed >= 5.0:
            self.fps = self.frames / elapsed
            self.get_logger().info(f'YOLO Green Ball FPS: {self.fps:.2f}')
            self.frames = 0
            self.start = time.perf_counter()
        cv2.putText(display, f'FPS: {self.fps:.1f}',
                    (10, 20), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 255), 2)
        cv2.putText(display, f'Green Balls: {count}',
                    (10, 45), cv2.FONT_HERSHEY_SIMPLEX,
```

```
                    0.5, (0, 255, 255), 2)
        cv2.imshow('PW4 - YOLO Green Ball Detection', display)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            self.timer.cancel()
            rclpy.try_shutdown()

    def destroy_node(self):
        cv2.destroyAllWindows()
        super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = YOLOGreenBallNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

D3. Launch the existing camera_node in Terminal 1 (not in the venv).

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/install/setup.bash
export ROS_DOMAIN_ID=30
ros2 run mindspark_robot camera_node
```

D4. Launch the new YOLO–HSV node in Terminal 2 (yolo_venv required).

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/yolo_venv/bin/activate
export ROS_DOMAIN_ID=30
python3 ~/mindspark_ws/src/mindspark_robot/mindspark_robot/yolo_green_ball_node.py
```

D5. Verify data in Terminal 3. Green-ball messages must match the PW3 String interface.

```
export ROS_DOMAIN_ID=30
source /opt/ros/jazzy/setup.bash
ros2 node list
ros2 topic list
ros2 topic info /camera/image_raw -v
ros2 topic echo /yolo/green_ball
ros2 topic hz /yolo/green_ball
```

Expected example: data: 'DETECTED: x=81, y=110'; when absent: data: 'NOT DETECTED'. Do not claim all non-green balls were rejected unless this is actually demonstrated.

SCREENSHOT EVIDENCE — D: YOLO green-only ROI / detection output / FPS

_____

_____

E. Register All Nodes in setup.py and Build

E1. The package already contains PW1–PW3 nodes. To avoid placement errors, use the complete final setup.py below. Preserve the actual maintainer metadata and the license in package.xml when applicable.

```
code ~/mindspark_ws/src/mindspark_robot/setup.py
```

```
#!/usr/bin/env python3
import os
from glob import glob
from setuptools import find_packages, setup

package_name = 'mindspark_robot'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'),
         glob('launch/*.launch.py')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ilmi',
    maintainer_email='ilmiariffin@gmail.com',
    description='MindSpark ROS2 Training Package',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'hello_node = mindspark_robot.hello_node:main',
            'simple_publisher = mindspark_robot.simple_publisher:main',
            'simple_subscriber = mindspark_robot.simple_subscriber:main',
            'camera_node = mindspark_robot.camera_node:main',
            'red_ball_node = mindspark_robot.red_ball_node:main',
            'green_ball_node = mindspark_robot.green_ball_node:main',
            'vision_decision_node = mindspark_robot.vision_decision_node:main',
            'esp32_control_node = mindspark_robot.esp32_control_node:main',
            'yolo_detection_node = mindspark_robot.yolo_detection_node:main',
            'yolo_sports_ball_node = mindspark_robot.yolo_sports_ball_node:main',
            'yolo_green_ball_node = mindspark_robot.yolo_green_ball_node:main',
        ],
    },
)
```

E2. Build without activating the YOLO venv, then check 11 executables (including simple_publisher and simple_subscriber from earlier work):

```
deactivate  # only if (yolo_venv) is currently active
cd ~/mindspark_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select mindspark_robot
source install/setup.bash
ros2 pkg executables mindspark_robot
ros2 pkg executables mindspark_robot | grep yolo
```

E3. Check generated shebang. It may point to system Python; registration alone does NOT ensure YOLO dependencies are available.

```
head -n 1 ~/mindspark_ws/install/mindspark_robot/lib/mindspark_robot/yolo_green_ball_node
```

E4. Run the installed ROS2 executable explicitly using venv Python:

```
~/mindspark_ws/yolo_venv/bin/python3 \
  ~/mindspark_ws/install/mindspark_robot/lib/mindspark_robot/yolo_green_ball_node
```

Expected: YOLO model loads, then waits for camera messages. Ctrl+C to stop. Note: this is NOT the same as plain ros2 run with a system-Python shebang.

SCREENSHOT EVIDENCE — E: build, executable registration and interpreter verification

_____

_____

F. Reuse Existing Decision and ESP32 Nodes

F1. Keep camera_node and yolo_green_ball_node running. Start decision node with topic remapping (no source-code changes):

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/install/setup.bash
export ROS_DOMAIN_ID=30
ros2 run mindspark_robot vision_decision_node \
  --ros-args -r /green_ball:=/yolo/green_ball
```

F2. Verify remapped input and decision output:

```
ros2 node info /vision_decision_node
ros2 topic echo /vision/decision
ros2 topic hz /vision/decision
```

Expected remapped subscription: /yolo/green_ball (std_msgs/msg/String). Expected output: L, C, R or N at 5 Hz. Only one decision publisher should operate at a time.

F3. Start ESP32 control node in a separate terminal (close Arduino Serial Monitor first):

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/install/setup.bash
export ROS_DOMAIN_ID=30
ros2 run mindspark_robot esp32_control_node
```

Green ball position	Decision	ESP32 output	Observed (tick)
LEFT	L	Red LED, GPIO25	☐
CENTER	C	Yellow LED, GPIO26	☐
RIGHT	R	Green LED, GPIO27	☐
NOT DETECTED	N	All LEDs OFF	☐

F4. Verify /vision/decision has one publisher and one intended ESP32 subscriber (counts increase while CLI echo/hz is running).

```
ros2 topic info /vision/decision -v
```

SCREENSHOT EVIDENCE — F: remapped decision and ESP32 hardware response

_____

_____

G. Create and Run the PW4 ROS2 Launch File

G1. Create launch file in the existing launch directory:

```
mkdir -p ~/mindspark_ws/src/mindspark_robot/launch
code ~/mindspark_ws/src/mindspark_robot/launch/pw4_yolo_hardware.launch.py
```

G2. Full Python launch file. The YOLO process is explicitly started with yolo_venv Python; all other nodes use ROS2 executables.

```
#!/usr/bin/env python3
import os
from launch import LaunchDescription
from launch.actions import ExecuteProcess
from launch_ros.actions import Node

def generate_launch_description():
    home = os.path.expanduser('~')
    yolo_python = os.path.join(
        home, 'mindspark_ws', 'yolo_venv', 'bin', 'python3')
    yolo_executable = os.path.join(
        home, 'mindspark_ws', 'install', 'mindspark_robot',
        'lib', 'mindspark_robot', 'yolo_green_ball_node')

    camera_node = Node(
        package='mindspark_robot', executable='camera_node',
        name='camera_node', output='screen')

    yolo_green_ball_node = ExecuteProcess(
        cmd=[yolo_python, yolo_executable], output='screen')

    vision_decision_node = Node(
        package='mindspark_robot', executable='vision_decision_node',
        name='vision_decision_node', output='screen',
        remappings=[('/green_ball', '/yolo/green_ball')])

    esp32_control_node = Node(
        package='mindspark_robot', executable='esp32_control_node',
        name='esp32_control_node', output='screen')

    return LaunchDescription([
        camera_node, yolo_green_ball_node,
        vision_decision_node, esp32_control_node
    ])
```

G3. Stop all manually launched nodes to avoid duplicate publishers and USB/camera resource conflicts. Build and launch from system ROS2 Python:

```
cd ~/mindspark_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select mindspark_robot
source install/setup.bash
export ROS_DOMAIN_ID=30
ros2 launch mindspark_robot pw4_yolo_hardware.launch.py
```

G4. In a new terminal, verify all four nodes and related topics:

```
source /opt/ros/jazzy/setup.bash
source ~/mindspark_ws/install/setup.bash
export ROS_DOMAIN_ID=30
ros2 node list
ros2 topic list
ros2 node info /vision_decision_node
ros2 topic hz /yolo/green_ball
ros2 topic echo /vision/decision
```

Reference observation from the development test: camera ≈30 FPS, YOLO–HSV ≈30 FPS, /yolo/green_ball ≈30 Hz. These are observed values, not guaranteed values on all machines. The screenshot confirmed L and N command logging; CENTER/RIGHT LED response is a participant test.

SCREENSHOT EVIDENCE — G: ROS2 launch window, 4 active nodes, topic rate, physical LED

_____

_____

H. Final Practical Verification

Test	Verification method	Result
H1	General YOLO displays multiple COCO object classes	☐ Pass ☐ Fail
H2	Sports ball node filters class ID 32	☐ Pass ☐ Fail
H3	YOLO–HSV accepts green sports ball	☐ Pass ☐ Fail
H4	Non-green sports ball rejected (where YOLO detects it)	☐ Pass ☐ Fail
H5	/yolo/green_ball publishes expected String message	☐ Pass ☐ Fail
H6	Remapped decision node emits L/C/R/N	☐ Pass ☐ Fail
H7	ESP32 LEDs react for L/C/R and N	☐ Pass ☐ Fail
H8	Launch file starts four nodes using correct Python env	☐ Pass ☐ Fail

Measured camera publishing FPS: _____ YOLO detection FPS: _____ /yolo/green_ball Hz: _____
/vision/decision Hz: _____

Observed limitations / remarks: _____

I. Reflection Questions

1. What does YOLO class ID 32 represent, and why does YOLO11n not directly classify green versus blue sports balls?

_____

2. Why is HSV filtering applied inside the YOLO bounding box rather than over the entire frame?

_____

3. Explain the roles of /camera/image_raw, /yolo/green_ball and /vision/decision.

_____

4. What does ROS2 topic remapping allow us to reuse from PW3?

_____

5. Why must the YOLO executable use the Python interpreter from yolo_venv on this setup?

_____

6. What happens if YOLO detects two green balls with different confidence values?

_____

7. How would intermittent missed detections affect LED behaviour, and what safeguard could be added?

_____

J. Completion Checklist

☐ YOLO environment ready ☐ General node tested ☐ Sports-ball node tested ☐ YOLO–HSV ROS2 node tested

☐ setup.py updated and built ☐ topic remapping confirmed ☐ ESP32 tested ☐ launch file tested ☐ screenshots attached

Instructor comments: _____

Participant signature: _____ Instructor verification: _____